Set & Map Interfaces

- **Duplicates are not allowed.**
- **Constant time search operations.** It means it gives the result immediately and fast.

Drawbacks of Set & Maps

- **Positional access**→ values kisi order me store nahi hoti ha, kahi bhi store ho skti ha.
- It can't perform index based searches, like List DataStructures.

Maps

- It stores→ key, values.
- Duplicate **values** are allowed, but….
- Duplicate **keys** are not allowed.
- **Constant** time search.

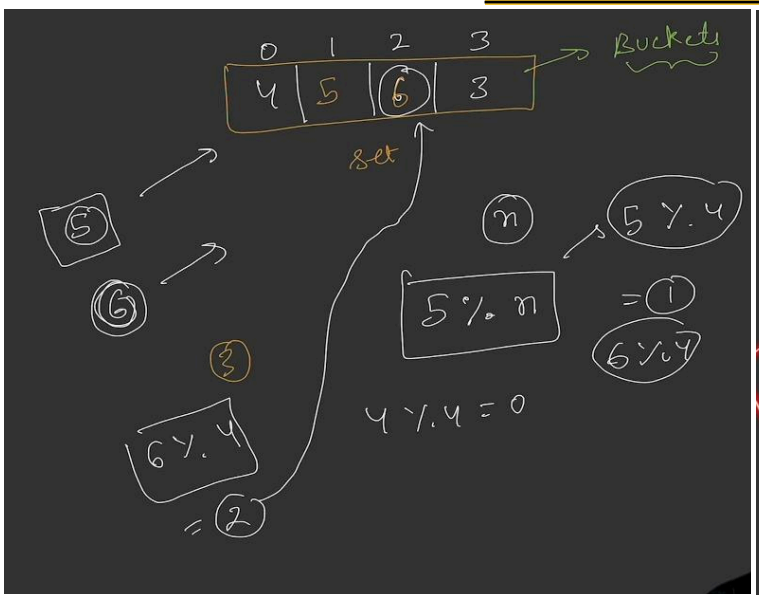
Example:

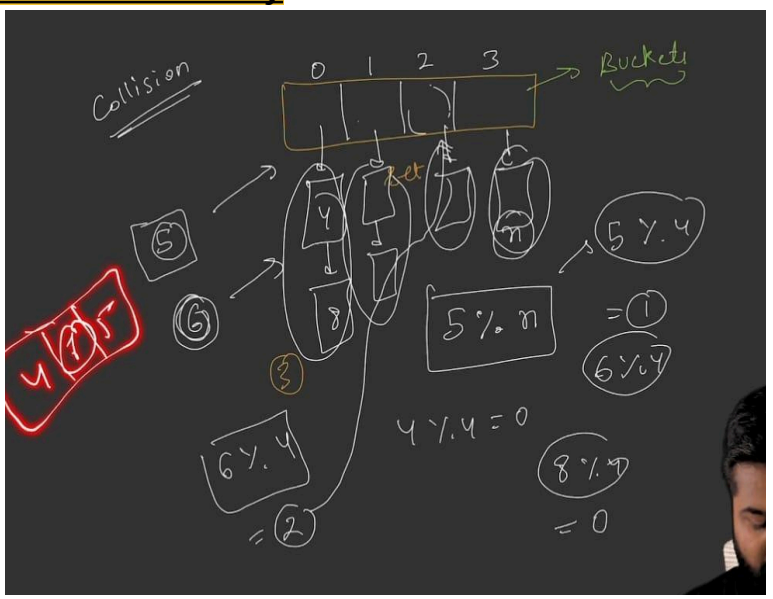
RollNumber and Students Names:--> here Student Names can have duplicate but RollNumbers can't be duplicates.

Methods of MAP INTERFACE

```
import java.util.*;
public class SetandMaps {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
            set.add("Aditya");
            set.add("Rohit");
            set.add("Rohan ");
                System.out.println(set.contains("Ronit"));
        Map<Integer, String> map = new HashMap<>();
            map.put(100, "Rahul");
            map.put(101,"Aryan");
            map.put(102, "Gaurav");
            map.put(103, "Rahul");
                System.out.println(map.containsKey(101));
                System.out.println(map.get(102 ));
    }
}
```

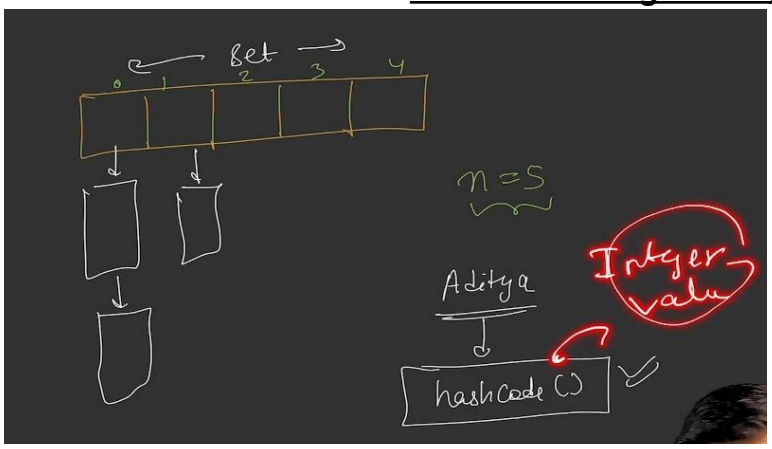
How Set values are stored internally

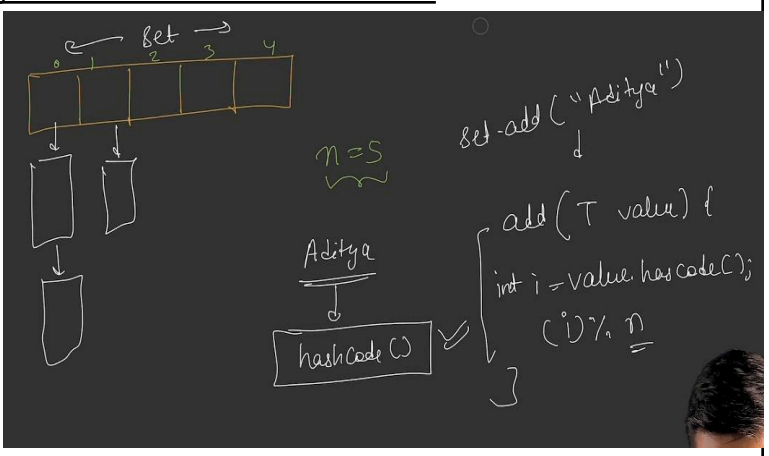




- **Kisi bhi value ko store krne ke lia set datastructure ke size ke sath module kr dete ha. Example→ 5%4=1 ans, it means 1st index.**
- **Or value ko nekalne ke liye us value ko fir se Set DataStructure ke size se module krana pdega, us se index value aaegi.**
- **Or ek hi index pr multiple values store krne ke liya, hum har index ko LinkedList me convert kr dete ha taki sabhi values particular index me save ho ske.**

In case of String values, Stored in Set DataStructure





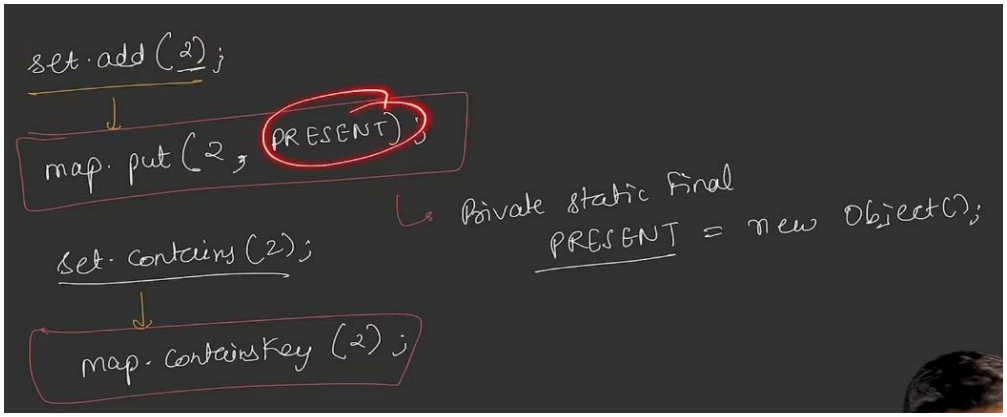
String values ka hum phle hashSet nekalte ha,
Or fir us hashSet ko set Datastructure ke size se module krte ha, taaki pta chle ki value kis bucket me jani chahiye.
Or baki process same hi rhta ha..

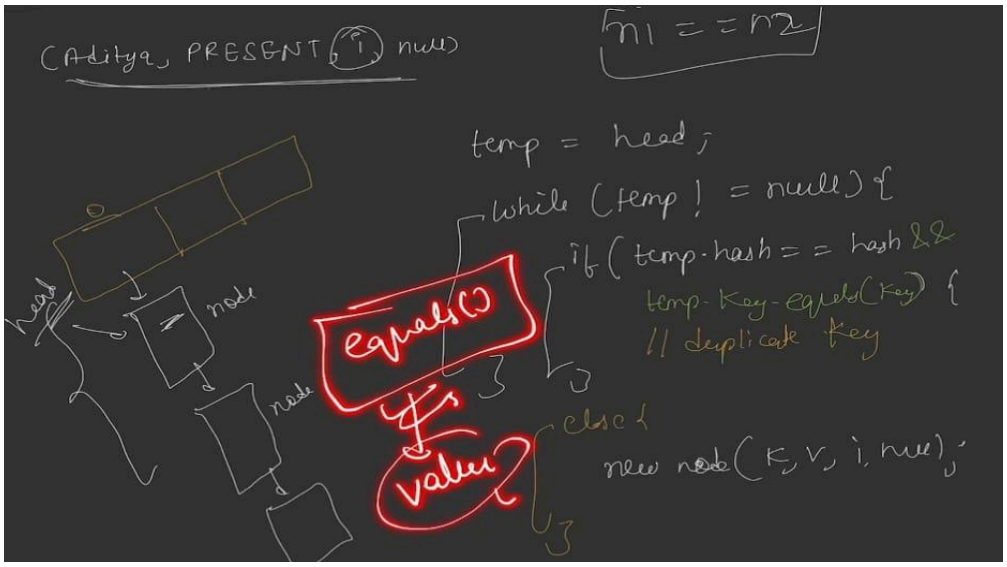
How Map Values are Stored Internally

Maps me bhi values same hi store hoti ha.
Isme values key ka hash set nikal ke store hoti ha.
Fir hashSet ko DataStructure ke size se module krte ha, taki pta chle ki value kis bucket me jani chahiye.

Java me **set** nam ka koi dataStructure **nahi** hota, jab bhi hum set bnate ha to **internally** java ek **Map** hi bna ke deta ha.







- To handle **collision** in **Map** & **set buckets** we use **linkedList** and this process is known as **chaining**.
- One more way to handle collision but it is not used in java language, it is known as **OPEN ADDRESSING**.
- OPEN ADDRESSING→ isme hum kisi bhi bucket ki repeated values ko next bucket me shift kr dete ha. Or jo us next bucket ki original value ha use bhi next me shift krna pdta ha, & this is very complex.

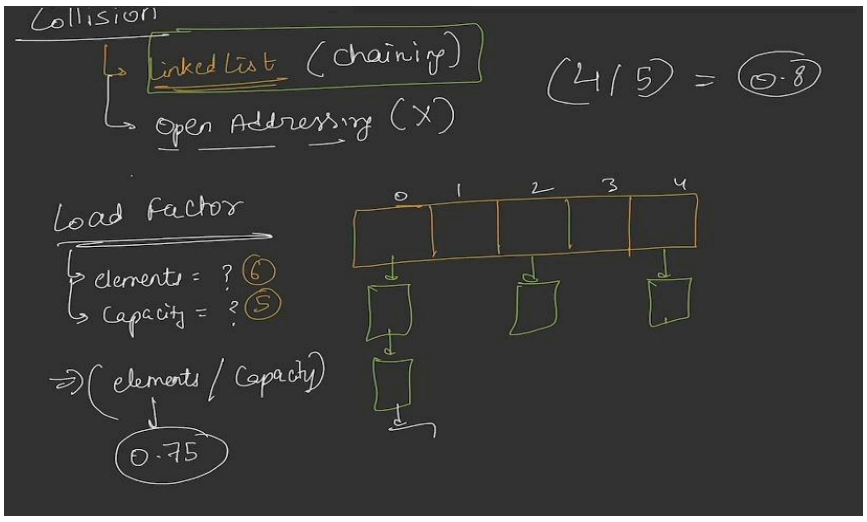
LOAD FACTOR in java

- How many number of elements ?
- How much capacity ?

We have a formula→ (elements / capacity)= if ans is >0.75 then it means ki hamare bucket ka array bhut full ha, it means 75% se zada full ha.

To us case me hum krege **Rehashing**→ ki hum existing array ko uske double size ke Array me dal dege.

- Isko krne ke piche ka simple reason ha, taki humari bucket me hume no of 1 times pr result mile, or load na pde.

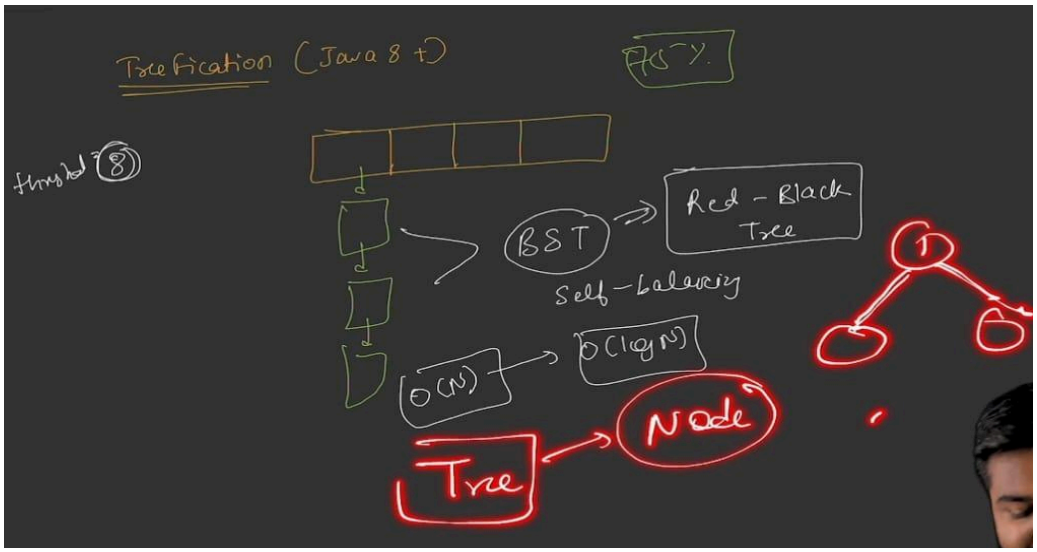


Steps of Re-Hashing.

- Create 2x as Array.
- Now again re-mod the keys of the array, and giving them new bucket indexings.
- Initial size of the array of Set & Map is 16 size, it means the initial capacity of the array is → 16.

Tree-fication (java 8+)

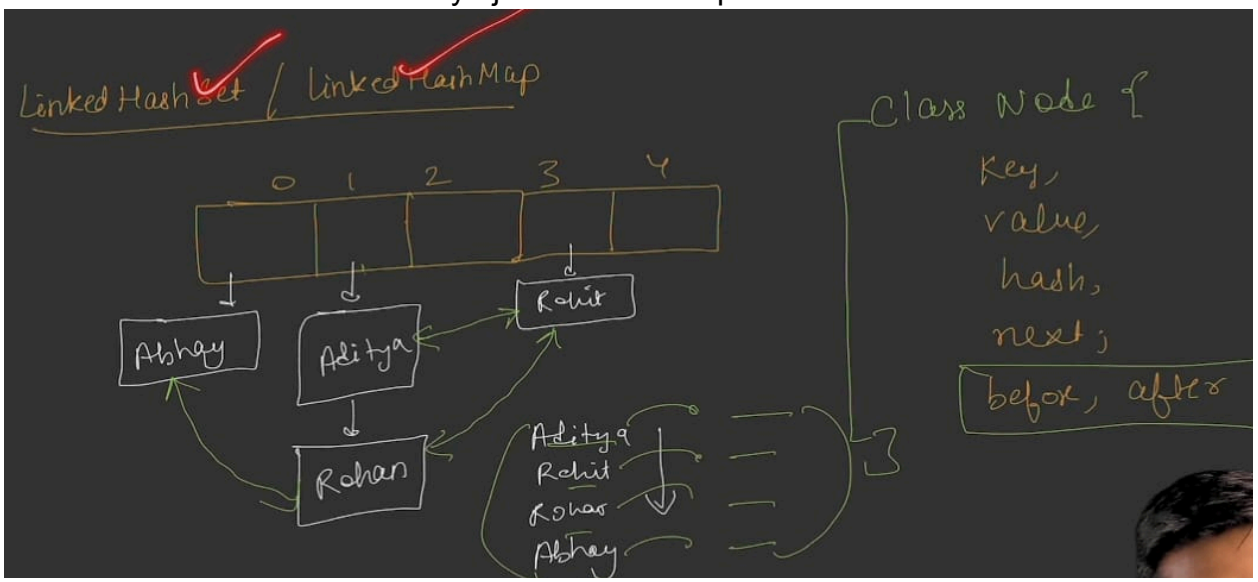
- Ye concept automatically internally hota ha jab bhi elements apni thrashold se zada hote ha.



- Kum elements ki collection ke liye hm prefer krte ha LinkedList or,
- Zada elements ki collection ke liye hum prefer krte ha Red-Black Tree.

Linked HashSet / Linked HashMap

- In me hume proper order milta ha.
- Doubly LinkedList is used in this.
- Is se dataStructure ka order preserved reheta ha.
- Ye thoda complex ha or sath hi ye storage zada leta ha.
- Iska use tabhi krna chahiye jab hume order preservation se matlab ho.



Tree-Map & Tree-Set

- Here we use (Self Balancing BST)--> (Red-Black Tree).
- Yaha pr hashMap wala concept kam nahi krta ha.
- Yaha pr hashing use nahi hoti ha.
- Yaha pr time complexity order of (log-n) ho jati ha.

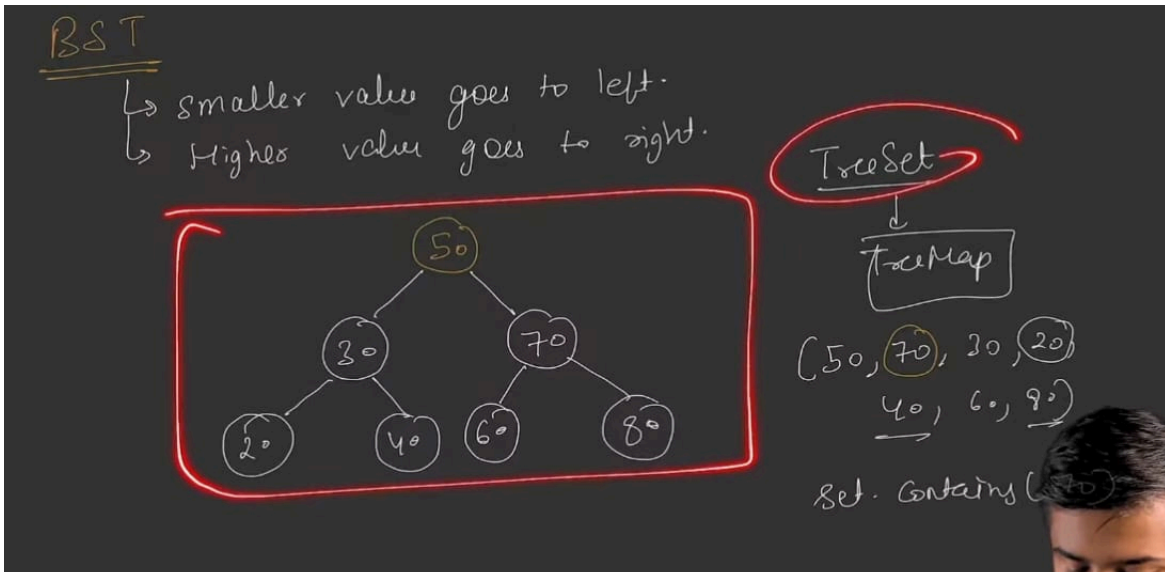
It unlocks many methods in dataStructure which can't be given by hashMap & Linked HashMap→

- Keys sorted order.
- Find the largest element.
- Find the smallest element.
- Range query…….give me numbers between 20 to 75.

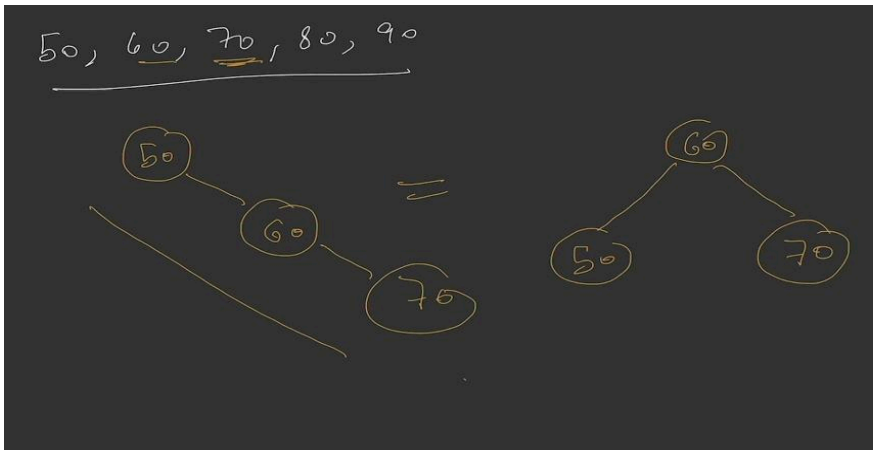
In case of strings these methods works on Lexicographical order→ dictionary order of Alphabets.

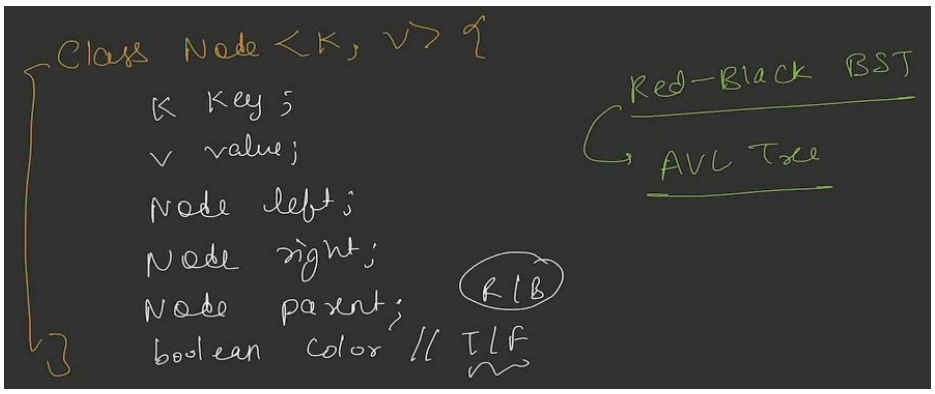
Concept of Normal BST

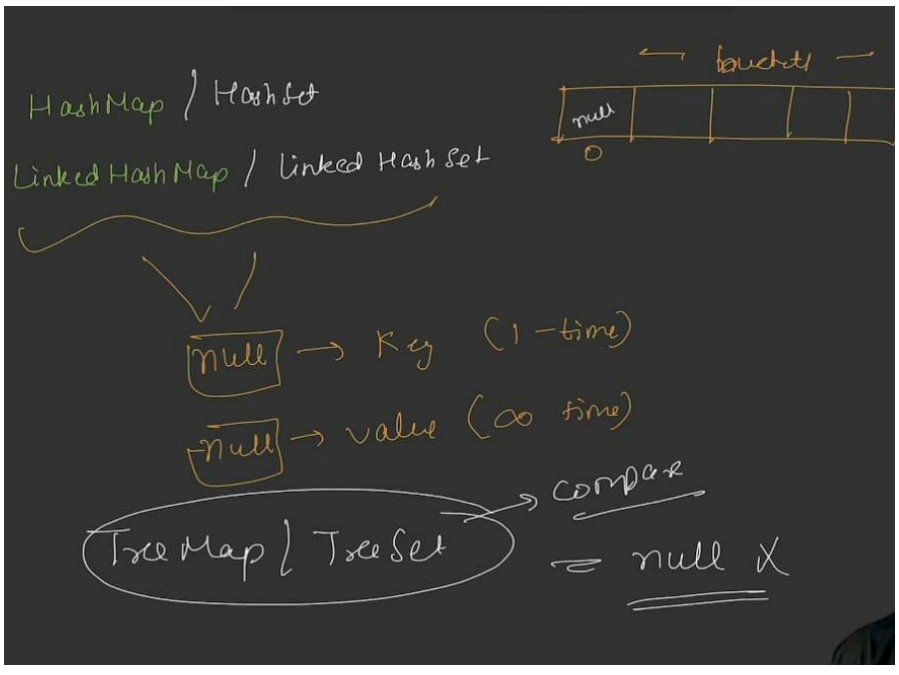
- Smaller value goes to left.
- Higher value goes to right.

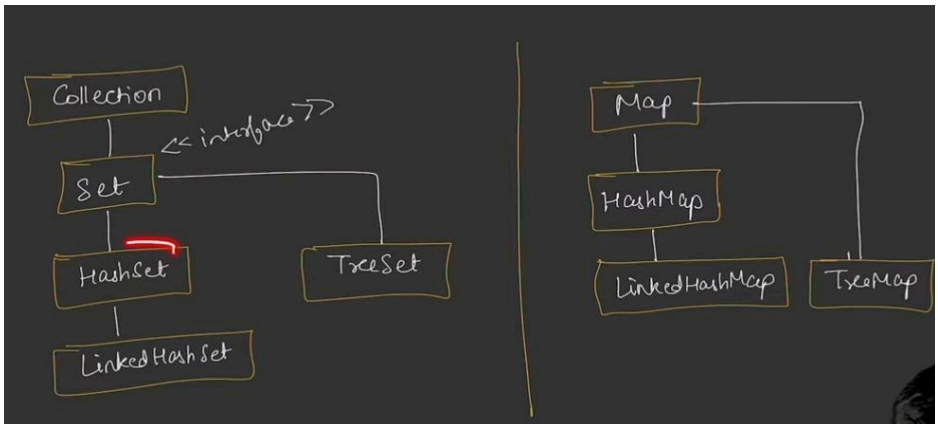


Concept of Self-Balancing BST








Collection
<< interface >>
Set
HashSet
TreeSet
LinkedHashSet
Map
HashMap
LinkedHashMap
TreeMap